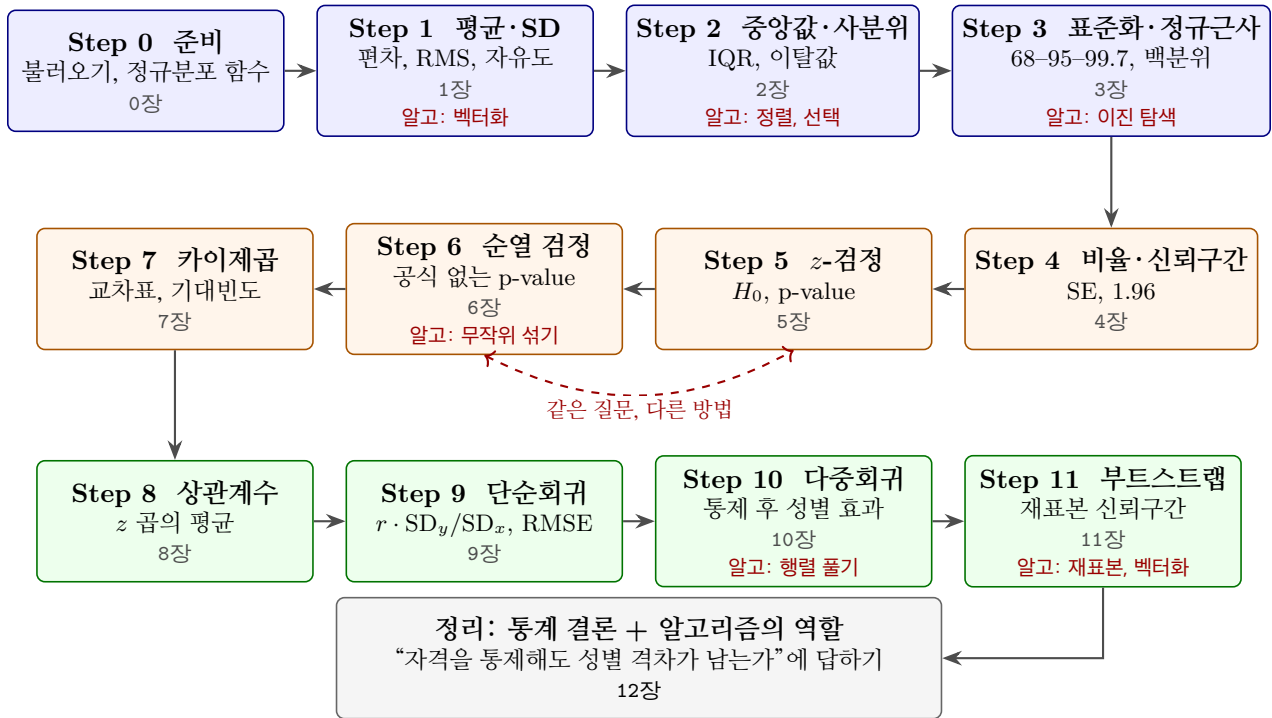


numpy·pandas로 직접 만드는 통계

통계 공식을 코드로 손수 구현하고, 알고리즘으로 결과를 더 낮게 만드는 연습

연습: stats_basics_practice.ipynb (빈칸) 정답: stats_basics_solution.ipynb

라이브러리: numpy, pandas, math



- | | |
|--|------------------------------------|
| 요약 (Step 0-3) | 한 변수의 중심과 퍼짐, 분포 모양 |
| 추정과 검정 (Step 4-7) | 그룹 차이를 추정하고, 우연인지 검증 |
| 관계와 회귀 (Step 8-11) | 변수 사이의 관계, 통제 후의 효과 |
| 알고: 알고리즘 포인트 | 정렬, 이진 탐색, 무작위 섞기, 재표본, 벡터화가 쓰이는 곳 |

이 워크북의 목표 (학부 1-2학년 수준)

- 통계를 탄탄하게: 공식을 외우는 대신, 한 줄씩 코드로 직접 계산해서 의미를 손으로 익힌다.
- pandas·numpy로 구현: 직접 만든 값이 pandas 내장 함수와 같은지 매번 확인한다.
- 알고리즘은 도움이 될 때만: 정렬·이진 탐색·무작위 섞기·재표본이 계산을 빠르게 하거나, 공식의 가정 없이 답을 주는 지점까지.

scipy, statsmodels, sklearn은 쓰지 않습니다. (확인용으로 다른 책에서만 등장)

목차 (Contents)

이 워크북 쓰는 법	3
0 Step 0. 준비: 데이터와 정규분포 함수	4
1 Step 1. 평균, 분산, 표준편차를 직접	5
2 Step 2. 중앙값과 사분위수: 정렬로	6
3 Step 3. 표준화, 정규근사, 백분위	7
4 Step 4. 그룹 비율과 신뢰구간	8
5 Step 5. 두 비율의 z -검정을 직접	9
6 Step 6. 순열 검정: 공식 없이 p-value	10
7 Step 7. 교차표와 카이제곱을 직접	11
8 Step 8. 상관계수를 직접	12
9 Step 9. 단순회귀: 공식 두 가지로	13
10 Step 10. 다중회귀: 행렬로 직접	14
11 Step 11. 부트스트랩: 재표본으로 신뢰구간	15
12 정리: 통계 결론과 알고리즘의 역할	16
12.1 통계 결론	16
12.2 알고리즘이 한 일	16
12.3 바꿔 보며 연습	16
13 확인 질문 답과 빈칸 정답	17
13.1 확인 질문	17
13.2 빈칸 정답	17

이 워크북 쓰는 법

1. VS Code에서 왼쪽에 이 PDF, 오른쪽에 `stats_basics_practice.ipynb`를 엽니다. 정답 노트북은 닫아 둡니다.
2. 각 장: 목표 읽기 → 노트북에서 `---` 채우고 실행 → 실행 결과와 비교 → 무슨 일이 일어났나 읽기 → 확인 질문.
3. 막히면 5분 고민 후 13장 끝 “빈칸 정답”을 봅니다.
4. 장이 끝날 때마다 숫자 하나를 바꿔서 결과가 어떻게 달라지는지 예상하고 확인합니다.

※ 작은 주석: **[흐름 n]** = pandas-stats-walkthrough, **[응용 n]** = hiring-statistics-book, **[알고 n]** = algorithms-for-datathon, **[Notes Def x]** = Statistics 1 강의노트. 더 깊이 보고 싶을 때만 따라가면 됩니다.

상자	역할
목표	이 단계에서 할 일, 통계 개념
코드	노트북과 같은 빈칸 코드
실행 결과	실제로 돌린 출력 (정답 노트북과 동일)
무슨 일이 일어났나	계산 과정을 단계별로
알고리즘이 더 낮게 만드는 점	알고리즘이 쓰이는 장애만
확인 질문	답은 13장

0 Step 0. 준비: 데이터와 정규분포 함수

목표

데이터를 불러오고, 표준정규 누적분포 $\Phi(z)$ 를 scipy 없이 만든다.

```
import math
import numpy as np
import pandas as pd

def Phi(z):
    """표준정규 누적분포 P(Z ≤ z). math.erf로 계산."""
    return 0.5 * (1 + math.erf(z / math.sqrt(2)))

df = pd.read_csv("hiring_practice.csv")
d = df.dropna()
print(df.shape, d.shape)
```

실행 결과

(600, 7) (582, 7)

무슨 일이 일어났나

1. df: 원본 600행. d: test_score가 빈 18행을 뺀 582행. 점수를 쓰는 계산은 d, 합격률만 쓰는 계산은 df를 씁니다.
2. $\Phi(z) = P(Z \leq z)$ 는 표준정규곡선 아래 z 왼쪽의 면적입니다 [\[흐름 3.2\]](#). 이 함수 하나로 p-value와 정규근사를 모두 계산합니다.
3. 왜 erf인가: $\Phi(z) = \frac{1}{2} \left(1 + \operatorname{erf}\left(\frac{z}{\sqrt{2}}\right) \right)$ 는 정규분포 적분을 표준 함수로 쓴 것입니다. 파이썬 math에 들어 있습니다.

※ 확인: $\Phi(1.96) = 0.975$, $\Phi(0) = 0.5$, $\Phi(-1) = 1 - \Phi(1) = 0.159$ (대칭).

확인 질문

1. $\Phi(2) - \Phi(-2)$ 는 무엇을 뜻하고, 값은 대략 얼마인가?

1 Step 1. 평균, 분산, 표준편차를 직접

목표

평균과 표준편차를 공식 그대로 계산하고, 분모 n 과 $n-1$ 의 차이(자유도)를 확인한다. [흐름 2.2]

[Notes Def 2.3, 2.5]

$$\bar{x} = \frac{1}{n} \sum x_i, \quad SD = \sqrt{\frac{1}{n} \sum (x_i - \bar{x})^2} \text{ (편차의 RMS)}, \quad s = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}.$$

```
x = d["test_score"].to_numpy()
n = len(x)
mean = x.__( ) / n
dev = x - __
sd_pop = math.sqrt((dev ** 2).sum() / __) # 분모 n
sd_sample = math.sqrt((dev ** 2).sum() / __) # 분모 n-1
print(round(mean, 3), round(sd_pop, 3), round(sd_sample, 3))
print(round(d["test_score"].std(), 3), round(np.std(x), 3))
```

실행 결과

```
65.235 11.647 11.657
11.657 11.647
```

무슨 일이 일어났나

1. `to_numpy()`: pandas 열을 numpy 배열로. 이제 순수한 숫자 582개입니다.
2. `x - mean`: 582개 모두에서 평균을 한 번에 뺍니다 (벡터 연산). 편차의 합은 0입니다.
3. 분모 n : 11.647, 분모 $n-1$: 11.657. **pandas** `.std()`는 $n-1$, **numpy** `np.std()`는 n 이 기본값이라 두 값이 다르게 나옵니다.
4. 왜 $n-1$ 인가: 편차는 합이 0이라는 제약이 하나 있어서 자유롭게 움직이는 값이 $n-1$ 개 (자유도). n 으로 나누면 분산을 평균적으로 작게 추정합니다 (bias) [흐름 2.2, 9.3].

알고리즘이 더 낮게 만드는 점

반복문 `for v in x: total += v`로도 같은 값이 나오지만, `x.sum()`과 `x - mean`은 C로 한 번에 처리되어 수십~수백 배 빠릅니다 (100만 개에서 약 230배) [알고 1]. 같은 $\Theta(n)$ 이라도 상수가 다릅니다. 행 단위 반복 대신 배열 연산이 pandas·numpy의 제1원칙입니다.

확인 질문

2. $n=5$ 일 때 n 과 $n-1$ 로 나눈 분산의 비는? $n=582$ 일 때는?

2 Step 2. 중앙값과 사분위수: 정렬로

목표

정렬로 중앙값, 사분위수, IQR, 이탈값을 직접 구한다. [흐름 2.1, 2.3] [Notes Def 2.1, 2.4, 2.9]

정렬한 값 $x_{(1)} \leq \dots \leq x_{(n)}$ (order statistics)에서, p -분위수는 $x_{(\lceil pn \rceil)}$ 로 정의할 수 있습니다 [Notes Prop 5.6].

```
s = np.___(x) # 정렬
median = (s[n // 2 - 1] + s[n // 2]) / 2 if n % 2 == 0 else s[n // 2]
q1 = s[math.ceil(0.25 * n) - 1]
q3 = s[math.ceil(0.75 * n) - 1]
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = s[(s < lo) | (s > hi)]
print(median, q1, q3, iqr, (lo, hi))
print(outliers)
print(np.partition(x, n // 2)[n // 2])
```

실행 결과

```
65.0 58.0 73.0 15.0 (35.5, 95.5)
[ 29.  32.  33.  34.  35.  35.  35.  98. 100. 100.]
65.0
```

무슨 일이 일어났나

1. $n = 582$ (짝수)라서 중앙값은 291번째와 292번째 값의 평균. 파이썬 인덱스는 0부터라 $s[290]$, $s[291]$.
2. Q_1 : $\lceil 0.25 \times 582 \rceil = 146$ 번째 값 = 58. Q_3 : $\lceil 0.75 \times 582 \rceil = 437$ 번째 값 = 73.
3. 울타리 $[58 - 22.5, 73 + 22.5] = [35.5, 95.5]$ 밖의 10개가 이탈값. 이 값들이 틀린 값인지는 따로 확인해야 하고, 100점은 가능한 점수이므로 지우지 않습니다 [흐름 2.3].
4. 평균 $65.2 \approx$ 중앙값 65 $\Rightarrow$ 거의 대칭인 분포.

알고리즘이 더 낮게 만드는 점

- 정렬 `np.sort`: introsort (quick sort + heap sort), $O(n \log n)$ [알고 2]. 분위수를 여러 개 구할 때는 한 번 정렬해 두고 인덱스로 꺼내는 것이 효율적입니다.
- 선택 `np.partition(x, k)`: k 번째 값만 제자리에 놓고 나머지는 대충 나눕니다. quickselect, 평균 $O(n)$ [알고 3]. 중앙값 하나만 필요하면 정렬보다 빠릅니다 (100만 개에서 21 ms vs 7 ms).

확인 질문

3. 데이터에 999점(입력 오류)이 하나 추가되면 평균과 중앙값 중 무엇이 크게 변하는가?

3 Step 3. 표준화, 정규근사, 백분위

목표

z -score를 만들고, 실제 비율과 정규분포 근사를 비교한다. 백분위는 이진 탐색으로 구한다. [흐름 3]

```
z = (x - ___) / ___
for k in [1, 2, 3]:
    actual = (np.abs(z) <= k).mean()
    normal = Phi(k) - Phi(___)
    print(k, round(actual, 3), round(normal, 3))
z80 = (80 - mean) / sd_sample
print(round((x >= 80).mean(), 3), round(1 - Phi(___), 3))
pos = np.___(s, 85, side="right") # 이진 탐색
print(pos, round(pos / n, 3), round(Phi((85 - mean) / sd_sample), 3))
```

실행 결과

```
1 0.682 0.683
2 0.955 0.954
3 0.998 0.997
0.107 0.103
557 0.957 0.955
```

무슨 일이 일어났나

1. `np.abs(z) <= k`: 582개의 True/False. `.mean()`은 True의 비율 (True = 1).
2. 실제 68.2 / 95.5 / 99.8% vs 정규 68.3 / 95.4 / 99.7%: 68-95-99.7 규칙이 거의 정확합니다. 점수는 정규분포로 근사해도 됩니다.
3. 80점 이상: 실제 10.7%, 정규근사 $1 - \Phi(1.27) = 10.3\%$.
4. 85점의 백분위: 정렬된 `s`에서 85 이하인 값이 557개 $\Rightarrow$ 95.7 백분위. 정규근사 95.5.

알고리즘이 더 낮게 만드는 점

`np.searchsorted(s, 85)`는 이진 탐색으로 “85가 들어갈 자리”를 $O(\log n)$ 에 찾습니다. `(x <= 85).sum()`은 $O(n)$. 값 하나면 차이가 없지만, 지원자 수만 명의 백분위를 모두 구하면 $O(n \log n)$ vs $O(n^2)$ 입니다. 전제조건: 배열이 정렬되어 있어야 합니다 (Step 2의 `s`) [알고 5].

확인 질문

4. `side="right"`를 `side="left"`로 바꾸면 결과가 왜 달라질 수 있나?

4 Step 4. 그룹 비율과 신뢰구간

목표

성별 합격률을 추정하고, 그 불확실성을 SE와 95% 신뢰구간으로 나타낸다. [흐름 4] [응용 5.5]

$$\hat{p} = \frac{k}{n}, \quad SE = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \quad 95\% \text{ CI} = \hat{p} \pm 1.96 \text{ SE}.$$

```
g = df.groupby(___)["hired"].agg(["sum", "count"])
g["p"] = g["sum"] / g["count"]
g["se"] = np.sqrt(g["p"] * (1 - g["p"]) / g["count"])
g["lo"] = g["p"] - 1.96 * g["se"]
g["hi"] = g["p"] + 1.96 * g["se"]
print(g.round(3))

diff = g.loc["M", "p"] - g.loc["F", "p"]
se_diff = math.sqrt(g.loc["M", "se"] ** 2 + g.loc["F", "se"] ** 2)
print(round(diff, 3), round(se_diff, 4), (round(diff - 1.96 * se_diff, 3), round(diff + 1.96 * se_diff, 3)))
```

실행 결과

	sum	count	p	se	lo	hi
F	68	313	0.217	0.023	0.172	0.263
M	125	287	0.436	0.029	0.378	0.493

0.218 0.0374 (0.145, 0.292)

무슨 일이 일어났나

1. `groupby("gender")`: 행을 F, M으로 나누고 각 그룹의 `hired`에서 합(합격자 수)과 개수(지원자 수)를 셉니다 [응용 0.5].
2. 새 열 `p`, `se`, `lo`, `hi`는 열끼리의 벡터 연산이라 두 그룹이 한 번에 계산됩니다.
3. 차이의 SE: 두 그룹이 독립이므로 분산을 더합니다. $\sqrt{0.023^2 + 0.029^2} = 0.037$.
4. 해석: 남녀 합격률 차이는 21.8%p로 추정되고, 95% 신뢰구간 [14.5, 29.2]%p는 0을 포함하지 않습니다.

※ 1.96은 $\Phi(1.96) = 0.975$, 즉 가운데 95%의 경계 [흐름 3.2]. 이 구간의 근거는 CLT입니다: $\hat{p}$ 는 표본이 크면 근사적으로 정규분포 [Notes §7.4].

확인 질문

5. 여성 표본이 4배(1252명)였다면 여성의 SE는 대략 얼마가 되나?

5 Step 5. 두 비율의 z -검정을 직접

목표

$H_0 : p_M = p_F$ 를 세우고, 검정통계량과 p-value를 직접 계산한다. [흐름 5.2, 11] [응용 5.2]

$$z = \frac{\hat{p}_M - \hat{p}_F}{\sqrt{\hat{p}(1 - \hat{p}) \left(\frac{1}{n_M} + \frac{1}{n_F} \right)}}, \quad \hat{p} = \frac{k_M + k_F}{n_M + n_F}, \quad p\text{-value} = 2(1 - \Phi(|z|)).$$

```
def two_prop_z(k1, n1, k2, n2):
    p1, p2 = k1 / n1, k2 / n2
    p = (k1 + k2) / (n1 + n2)  # H0에서 합친 비율
    se0 = math.sqrt(p * (1 - p) * (1 / n1 + 1 / n2))
    z = (p1 - p2) / se0
    pval = 2 * (1 - Phi(abs(z)))  # 양측
    return z, pval

z, pval = two_prop_z(125, 287, 68, 313)
print(round(z, 3), round(pval, 4))

for dept, sub in df.groupby("department"):
    k = sub.groupby("gender")["hired"].sum()
    m = sub.groupby("gender")["hired"].count()
    zz, pp = two_prop_z(k["M"], m["M"], k["F"], m["F"])
    print(dept, round(zz, 3), round(pp, 4))
```

실행 결과

```
5.718 1.0780230708107297e-08
Engineering 3.956 0.0001
Marketing -1.575 0.1153
```

무슨 일이 일어났나

1. 가설: H_0 “합격 확률은 성별과 무관”, H_1 “다르다” (양측).
2. H_0 가 참이면 두 그룹의 p 가 같으므로, 합친 비율 $193/600 = 0.322$ 로 SE를 계산합니다 (Step 4의 CI용 SE와 다른 점).
3. $z = 0.218/0.0382 = 5.72$: 차이가 SE의 5.7배. $p = 1.1 \times 10^{-8} \Rightarrow H_0$ 기각.
4. 부서별: Engineering은 기각 ($p = 0.0001$), Marketing은 기각 못 함 ($p = 0.115$). “차이가 없다”가 아니라 “증거가 부족하다”입니다. Marketing 남성은 61명뿐입니다 [흐름 11.4].
5. for dept, sub in df.groupby(...): 그룹별 부분 표를 하나씩 꺼내 같은 함수를 적용합니다.

확인 질문

6. 이 검정은 어떤 가정 위에 서 있나? (힌트: Φ 를 쓴 이유)

6 Step 6. 순열 검정: 공식 없이 p-value

목표

H_0 “성별과 합격은 무관”이 참이라면 성별 라벨을 섞어도 차이가 비슷해야 한다. 이 생각을 컴퓨터로 직접 실험한다. [흐름 12 ⑩]

```
rng = np.random.default_rng(0)
def perm_test(sub, reps=10_000):
    y = sub["hired"].to_numpy()
    is_m = (sub["gender"] == "M").to_numpy()
    obs = y[is_m].mean() - y[~is_m].mean()
    count = 0
    for _ in range(reps):
        shuffled = rng.__(is_m)
        diff = y[shuffled].mean() - y[~shuffled].mean()
        if abs(diff) >= abs(____):
            count += 1
    return obs, count / ____
print(perm_test(df))
print(perm_test(df[df["department"] == "Marketing"]))
```

라벨 섞기

실행 결과

```
(0.21828767352027695, 0.0)
(-0.08473793581159086, 0.1261)
```

무슨 일이 일어났나

1. `is_m`: 남성이면 True인 배열. `~is_m`은 반대 (여성).
2. `rng.permutation(is_m)`: True/False의 개수는 그대로 두고 순서만 섞습니다. “합격 여부는 그대로, 성별만 무작위로 다시 붙이기”.
3. 이렇게 1만 번 섞어서 만든 차이들이 H_0 에서의 분포입니다. 관측 차이보다 극단적인 비율이 p-value.
4. 전체: 1만 번 중 0번 $\Rightarrow p < 0.0001$ (z -검정 10^{-8} 과 같은 결론).
5. Marketing: $p = 0.126$ (z -검정 0.115와 거의 같음).

알고리즘이 더 낫게 만드는 점

- 가정이 없다: z -검정은 “표본이 커서 정규근사가 맞다”(CLT)를 가정합니다. 순열 검정은 데이터의 섞기만 하므로 그 가정이 필요 없어서, 작은 그룹이나 공식이 없는 통계량(중앙값 차이, 비율의 비 등)에도 그대로 씁니다.
- 알고리즘: `permutation`은 Fisher-Yates 셔플, $O(n)$. 총 비용 $O(\text{reps} \times n) = 10^4 \times 600$.
- 확인용으로 최고: 두 방법이 같은 답을 주면 결론을 더 믿을 수 있습니다. 첫 페이지 다이어그램의 “같은 질문, 다른 방법”.

확인 질문

7. `reps=10_000`에서 `count`가 0이면 p-value를 “0”이라고 보고해도 되는가?

7 Step 7. 교차표와 카이제곱을 직접

목표

“성별과 합격은 독립인가”를 교차표의 관측빈도와 기대빈도로 검정한다. [흐름 5.3, 12 ⑧] [응용 5.3]

$$E_{ij} = \frac{(\text{행 합}_i)(\text{열 합}_j)}{n}, \quad \chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

```
O = pd.crosstab(df["gender"], df["hired"]).to_numpy()
row = O.sum(axis=1)
col = O.sum(axis=0)
E = np.outer(row, col) / O.sum()
chi2 = ((O - E) ** 2 / E).sum()
pval = 2 * (1 - Phi(math.sqrt(chi2)))
print(O); print(E.round(1)); print(round(chi2, 2), pval)
```

실행 결과

```
[[245  68]
 [162 125]]
[[212.3 100.7]
 [194.7  92.3]]
32.7 1.0780230708107297e-08
```

무슨 일이 일어났나

1. axis=1: 가로로 합 (행 합 = 성별별 인원 313, 287). axis=0: 세로로 합 (열 합 = 불합격 407, 합격 193).
2. np.outer(row, col): 행 합 × 열 합을 모든 칸에 대해 만든 2 × 2 행렬. /600을 하면 “독립이라면 나왔을 인원”.
3. 여성 합격 칸: 관측 68, 기대 100.7. 독립이라면 약 101명이 붙었어야 하는데 68명.
4. $\chi^2 = 32.7$. 2 × 2 표에서는 $\chi^2 = z^2$ ($5.718^2 = 32.7$)이므로 p-value도 Step 5와 정확히 같습니다.

※ 2 × 2보다 큰 표(예: 학위 3종 × 합격)는 자유도가 2 이상이라 이 Φ 트릭을 쓸 수 없습니다. 그때는 순열 검정(Step 6)으로 p-value를 구할 수 있습니다. 라벨을 섞을 때마다 χ^2 를 다시 계산.

확인 질문

8. 기대빈도 행렬 E의 행 합과 열 합은 O의 것과 같은가? 왜?

8 Step 8. 상관계수를 직접

목표

r = “두 변수의 z -score를 곱해서 평균낸 것”을 코드로 확인한다. [흐름 6.2]

```
def corr(a, b):
    za = (a - a.mean()) / a.std()
    zb = (b - b.mean()) / b.std()
    return (za * ____).____()
    # numpy std = 분모 n

xs = d["test_score"].to_numpy()
ys = d["hired"].to_numpy()
ex = d["years_experience"].to_numpy()
print(round(corr(xs, ys), 4), round(corr(ex, ys), 4), round(corr(xs, ex), 4))
print(d[["test_score", "years_experience", "hired"]].corr().round(4))
```

실행 결과

```
0.2465 0.1971 0.0077
test_score years_experience hired
test_score      1.0000      0.0077 0.2465
years_experience 0.0077      1.0000 0.1971
hired           0.2465      0.1971 1.0000
```

무슨 일이 일어났나

1. 점수가 평균보다 높고 동시에 합격(평균보다 높음)이면 곱이 양수, 엇갈리면 음수. 평균이 양수 0.2465 $\Rightarrow$ 약한 양의 상관.
2. 직접 계산한 값과 `df.corr()`가 소수 넷째 자리까지 같습니다.
3. 점수-경력 $r = 0.008$: 두 자격 변수는 거의 무관. 그래서 회귀에서 서로 효과를 빼앗지 않습니다 (Step 10).

※ 여기서는 분모 n 인 numpy std를 써야 정확히 맞습니다. 분모 $n - 1$ 을 쓰려면 곱의 합을 $n - 1$ 로 나눠야 합니다. 둘을 섞으면 r 이 미세하게 틀립니다.

확인 질문

9. 점수를 10으로 나눠도 (`xs / 10`) r 이 그대로인 이유는?

9 Step 9. 단순회귀: 공식 두 가지로

목표

회귀 기울기를 (1) $r \cdot SD_y / SD_x$ 와 (2) 정규방정식으로 각각 구해서 같은지 확인하고, RMSE 공식을 검증한다. [흐름 7.2, 8.2] [Notes Thm 11.4]

```
r = corr(xs, ys)
slope = r * ys.std() / ___ # 방법 1
intercept = ys.mean() - slope * ___
X = np.column_stack([np.ones(len(xs)), xs]) # 방법 2: (X^T X) b = X^T y
b = np.linalg.solve(X.T @ X, X.T @ ___)
resid = ys - (intercept + slope * xs)
rmse = math.sqrt((resid ** 2).mean())
print(round(slope, 5), round(intercept, 4), b.round(5))
print(round(rmse, 4), round(math.sqrt(1 - r ** 2) * ys.std(), 4))
```

실행 결과

```
0.00991 -0.3218 [-0.32181  0.00991]
0.4538 0.4538
```

무슨 일이 일어났나

1. 방법 1: “ x 가 1 SD 늘면 y 는 r SD 늘어난다”. $0.2465 \times 0.468 / 11.65 = 0.00991$. 점수 1점당 합격 확률 +1%p.
2. 방법 2: `np.column_stack`으로 [1, 점수] 두 열짜리 X (582×2)를 만들고 $X^T X b = X^T y$ 를 풉니다. `@`는 행렬 곱. 같은 답 [절편, 기울기].
3. 잔차의 $RMS = 0.4538 = \sqrt{1 - r^2} \cdot SD_y$. 점수를 알아도 예측 오차는 $\sqrt{1 - 0.061} = 0.97$ 배, 3%만 줄어듭니다 [흐름 8.2].

※ `np.linalg.inv`로 역행렬을 구해서 곱해도 되지만, `solve`가 더 빠르고 수치적으로 안정적입니다 (가우스 소거). 선형대수 수업의 내용 그대로입니다.

확인 질문

10. $r = 0$ 이면 회귀선은 어떤 모양이고, RMSE는 무엇과 같은가?

10 Step 10. 다중회귀: 행렬로 직접

목표

학위·점수·경력·부서를 통제한 뒤에도 성별 효과가 남는지, 계수·SE· t 를 행렬 공식으로 직접 계산한다. [흐름 13] [응용 6]

$$\hat{\beta} = (X^T X)^{-1} X^T y, \quad \hat{\sigma}^2 = \frac{\sum e_i^2}{n - k}, \quad SE(\hat{\beta}_j) = \sqrt{\hat{\sigma}^2 [(X^T X)^{-1}]_{jj}}, \quad t = \frac{\hat{\beta}_j}{SE}.$$

```
cats = d[["gender", "department", "degree"]]
Xdf = pd.get_dummies(cats, drop_first=___).astype(float)
Xdf["test_score"] = d["test_score"]
Xdf["years_experience"] = d["years_experience"]
Xdf.insert(0, "const", 1.0)
X = Xdf.to_numpy(); y = d["hired"].to_numpy()
XtX = X.T @ X
beta = np.linalg.solve(XtX, X.T @ y)
resid = y - X @ beta
n, k = X.shape
sigma2 = (resid ** 2).sum() / (___) # 자유도
se = np.sqrt(np.diag(sigma2 * np.linalg.inv(XtX)))
t = beta / se
out = pd.DataFrame({"coef": beta, "se": se, "t": t,
                    "p": [2 * (1 - Phi(abs(v))) for v in t]}, index=Xdf.columns)
print(out.round(4))
```

실행 결과

	coef	se	t	p
const	-0.4528	0.1133	-3.9982	0.0001
gender_M	0.0880	0.0396	2.2199	0.0264
department_Marketing	-0.2214	0.0404	-5.4867	0.0000
degree_Master	0.0837	0.0375	2.2329	0.0256
degree_PhD	0.1200	0.0611	1.9641	0.0495
test_score	0.0098	0.0015	6.5207	0.0000
years_experience	0.0339	0.0090	3.7537	0.0002

무슨 일이 일어났나

1. `pd.get_dummies(..., drop_first=True)`: 범주를 0/1 열로. 기준 범주(F, Engineering, Bachelor)는 열이 없습니다 [응용 0.8].
2. `insert(0, "const", 1.0)`: 맨 앞에 절편용 1의 열. X 는 582×7 .
3. 자유도 $n - k = 582 - 7 = 575$: 계수 7개를 데이터로 추정했기 때문 (단순회귀의 $n - 2$ 와 같은 원리) [Notes Prop 11.13].
4. 결론: 다른 모든 변수를 같게 맞춰도 남성은 합격 확률이 8.8%p 높고 ($t = 2.22$, $p = 0.026$), 단순 비교의 21.8%p보다 크게 줄었습니다. 줄어든 부분은 주로 부서 (Marketing -22%p) 때문입니다.

※ 응용 책 m2의 robust SE는 0.0386이고, 여기서는 고전적인 SE 0.0396입니다. 0/1 결과에서는 robust SE가 원칙이지만 [응용 6.5], 결론은 같습니다. p-value는 n 이 커서 $t(575) \approx N(0, 1)$ 로 Φ 를 썼습니다.

확인 질문

11. `drop_first=False`로 하면 어떤 문제가 생기나? (힌트: 상수항 열과의 관계)

11 Step 11. 부트스트랩: 재표본으로 신뢰구간

목표

공식 없이, 데이터를 다시 뽑는 실험으로 Step 4의 신뢰구간을 재현한다. [흐름 9.2]

```
rng = np.random.default_rng(0)
y_all = df["hired"].to_numpy()
m_all = (df["gender"] == "M").to_numpy()
B, N = 5000, len(df)

idx = rng.integers(0, ___, size=(B, N)) # 복원추출 인덱스
yy, mm = y_all[idx], m_all[idx]
gaps = ((yy * mm).sum(axis=1) / mm.sum(axis=1)
        - (yy * ~mm).sum(axis=1) / (~mm).sum(axis=1))
print(round(gaps.std(), 4), np.percentile(gaps, [___, ___]).round(3))
```

실행 결과

0.0376 [0.145 0.291]

무슨 일이 일어났나

1. `rng.integers(0, N, size=(B, N))`: 0-599 중에서 600개를 복원추출로 뽑는 것을 5000번. 결과는 5000×600 정수 행렬. 한 행 = 가상의 “또 다른 600명 표본”.
2. `y_all[idx]`: 그 인덱스로 합격 여부를 꺼낸 5000×600 행렬. `mm`도 같은 방식으로 성별.
3. `(yy * mm).sum(axis=1) / mm.sum(axis=1)`: 행마다 “남성 중 합격 비율”. 여성도 같은 방식. 그 차이가 표본마다의 격차.
4. 5000개 격차의 SD $0.0376 \approx$ 공식 SE 0.0374 , 가운데 95% $[0.145, 0.291] \approx$ 공식 CI $[0.145, 0.292]$.

알고리즘이 더 낮게 만드는 점

- 재표본 = 표본분포를 직접 만드는 알고리즘: 공식(CLT)이 없는 통계량에도 그대로 씁니다. 예: 중앙값 차이, disparate impact ratio p_F/p_M , 회귀 계수.
- 벡터화: for문 5000번 대신 5000×600 행렬 한 번으로 처리. Step 6의 순열 검정도 같은 방식으로 바꿀 수 있습니다 [알고 1, 10].
- 비용 $O(B \cdot N)$: B 를 4배로 하면 시간 4배, 정밀도는 2배 ($\sqrt{B}$ 법칙).

확인 질문

12. 순열 검정(Step 6)과 부트스트랩(Step 11)은 둘 다 “섞는다”. 무엇이 다른가?

12 정리: 통계 결론과 알고리즘의 역할

12.1 통계 결론

질문	결과	단계
점수 분포는?	평균 65.2, SD 11.7, 거의 정규 (68-95-99.7 일치)	1-3
합격률 차이는?	남 43.6% vs 여 21.7%, 차이 21.8%p, CI [14.5, 29.2]	4
우연인가?	$z = 5.72$, $p \approx 10^{-8}$; 순열 $p < 10^{-4}$; $\chi^2 = 32.7$	5-7
부서별로는?	Engineering 유의 ($p = 0.0001$), Marketing 증거 부족 ($p \approx 0.12$)	5-6
자격 변수와의 관계	점수 $r = 0.25$, 경력 $r = 0.20$, 서로는 무관	8-9
통제 후 성별 효과	+8.8%p, $t = 2.22$, $p = 0.026$	10
결과는 안정적인가	부트스트랩 CI가 공식 CI와 일치	11

한 문장: “남성의 합격률이 21.8%p 높고, 시험점수·경력·학위·부서를 통제해도 약 9%p의 격차가 남는다. 이 결론은 정규근사 검정, 순열 검정, 부트스트랩에서 모두 같다.” (부서별 차이와 인과의 한계는 [응용 6.9, 9.2])

12.2 알고리즘이 한 일

알고리즘	어디서	더 나아진 점
벡터화	Step 1, 3, 11	반복문보다 수백 배 빠름
정렬	Step 2	중앙값, 사분위수, 이탈값을 한 번에
선택 (quickselect)	Step 2	중앙값 하나는 정렬 없이 $O(n)$
이진 탐색	Step 3	백분위를 $O(\log n)$ 에
무작위 섞기	Step 6	정규근사 가정 없는 p-value
행렬 풀기 (가우스 소거)	Step 9, 10	역행렬보다 빠르고 안정적
재표본	Step 11	공식 없는 신뢰구간

12.3 바뀐 보며 연습

바꿀 것	먼저 예상해 볼 것
Step 2에서 <code>x</code> 에 999 추가	평균, SD, 중앙값, IQR 중 무엇이 크게 변하나
Step 4를 <code>groupby("department")</code> 로	부서 간 차이의 CI는?
Step 5를 학위별 (Master vs Bachelor)로	학위 효과는 유의한가
Step 6의 <code>reps</code> 를 1000, 100000으로	p-value가 얼마나 흔들리나
Step 10에서 <code>department</code> 열 제거	성별 계수가 커지나 (omitted variable bias [응용 6.8])
Step 11에서 Marketing만으로	격차의 CI가 0을 포함하나

13 확인 질문 답과 빈칸 정답

13.1 확인 질문

- 표준정규에서 -2 와 2 사이 면적 = 평균 ± 2 SD 안에 들어갈 확률, 약 0.954.
- $n/(n-1)$: $n=5$ 이면 1.25 (25% 차이), $n=582$ 이면 1.0017 (무시할 만함).
- 평균. 중앙값은 순서만 보므로 거의 그대로이고, 평균은 $(999 - 65)/583 \approx 1.6$ 점 오릅니다. SD는 크게 커집니다.
- "right"는 85와 같은 값들의 오른쪽 위치 (85 이하인 개수), "left"는 왼쪽 위치 (85 미만인 개수). 85점인 사람이 있으면 그 수만큼 다릅니다.
- $SE \propto 1/\sqrt{n}$ 이므로 절반: 약 0.012.
- 독립인 관측, 그리고 표본이 커서 $\hat{p}$ 의 분포가 정규분포로 근사된다는 가정 (CLT). 그래서 Φ 로 p-value를 계산할 수 있습니다.
- 아니요. "1만 번 중 한 번도 없었다"이므로 $p < 1/10000$ 으로 보고합니다. 진짜 확률이 0이라는 뜻이 아닙니다.
- 같습니다. $\sum_j E_{ij} = \text{행합}_i$, $\frac{\sum_j \text{열합}_j}{n} = \text{행합}_i$. 기대빈도는 합계를 유지한 채 "독립"만 가정한 표입니다.
- z -score는 단위와 무관합니다: $x/10$ 의 평균과 SD도 $1/10$ 이 되어 z 가 그대로. r 은 z 끼리의 곱이므로 그대로입니다.
- 기울기 0, 높이 $\bar{y}$ 인 수평선. $RMSE = \sqrt{1-0} \cdot SD_y = SD_y$: x 가 예측에 아무 도움이 안 됩니다.
- 각 범주의 열을 모두 넣으면 (예: `gender_F + gender_M = 1`) 상수항 열과 정확히 같아져서 열들이 일차종속이 됩니다. $X^T X$ 의 역행렬이 없어 `solve`가 실패합니다 (dummy variable trap) [응용 6.4].
- 순열 검정은 H_0 (차이 없음)을 가정하고 라벨을 섞어 "우연이라면 얼마나 나오나"를 봅니다 $\rightarrow$ p-value. 부트스트랩은 가정 없이 행 전체를 복원추출해 "추정값이 얼마나 흔들리나"를 봅니다 $\rightarrow$ 신뢰 구간.

13.2 빈칸 정답

Step	빈칸
1	sum, mean, n, n - 1
2	sort, 0.25, 0.75, 1.5, 1.5
3	mean, sd_sample, -k, z80, searchsorted
4	"gender", "count", 1.96, 1.96
5	n1 + n2, se0, z
6	permutation, obs, reps
7	1, 0, outer, E
8	zb, mean
9	xs.std(), xs.mean(), ys
10	True, y, n - k, se
11	N, 2.5, 97.5

※ 전체 정답 코드와 실행 결과는 `stats_basics_solution.ipynb`에 있습니다.